

Chapter 55

LED PWM Controller (LEDC)

55.1 Overview

The LED PWM Controller is a peripheral designed to generate PWM signals for LED control. It has specialized features such as automatic duty cycle fading. However, the LED PWM Controller can also be used to generate PWM signals for other purposes.

55.2 Features

The LED PWM Controller has the following features:

- Eight independent PWM generators (i.e., eight channels)
- Maximum PWM duty cycle resolution: 20 bits
- Four independent timers that support fractional division
- Adjustable phase of PWM signal output
- PWM duty cycle dithering
- Automatic duty cycle fading — gradual increase/decrease of a PWM's duty cycle without interference from the processor. An interrupt will be generated upon fade completion
- Up to 16 duty cycle ranges for each PWM generator to generate gamma curve signals - each range can be independently configured in terms of fading direction (increase or decrease), fading amount (the amount by which the duty cycle increases or decreases each time), the number of fades (how many times the duty cycle fades in one range), and fading frequency
- PWM signal output in low-power mode (Light-sleep mode)
- Event generation and task response related to the Event Task Matrix (ETM) peripheral

Note that the four timers are identical regarding their features and operation. The following sections refer to the timers collectively as Timer x (where x ranges from 0 to 3). Likewise, the eight PWM generators are also identical in features and operation, and thus are collectively referred to as PWM n (where n ranges from 0 to 7).

55.3 Architectural Overview

Figure 55.3-1 shows the architecture of the LED PWM Controller.

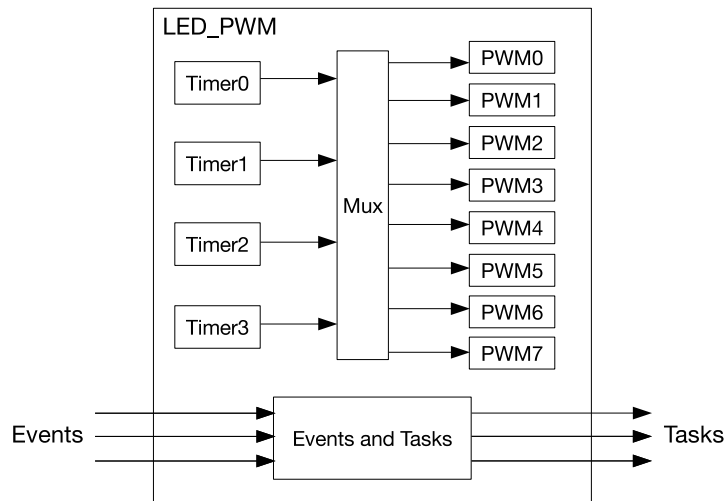


Figure 55.3-1. LED PWM Architecture

Each of the four timers has an internal timebase counter (i.e., a counter that counts on cycles of a reference clock) and thus can be independently configured (i.e., configurable clock divider, and counter overflow value). Each PWM generator selects one of the timers by configuring the `LEDC_TIMER_SEL_CHn`, and uses the timer's counter value `timerx_cnt` as a reference to generate its PWM signal.

Figure 55.3-2 illustrates the main functional blocks of the timer and the PWM generator.

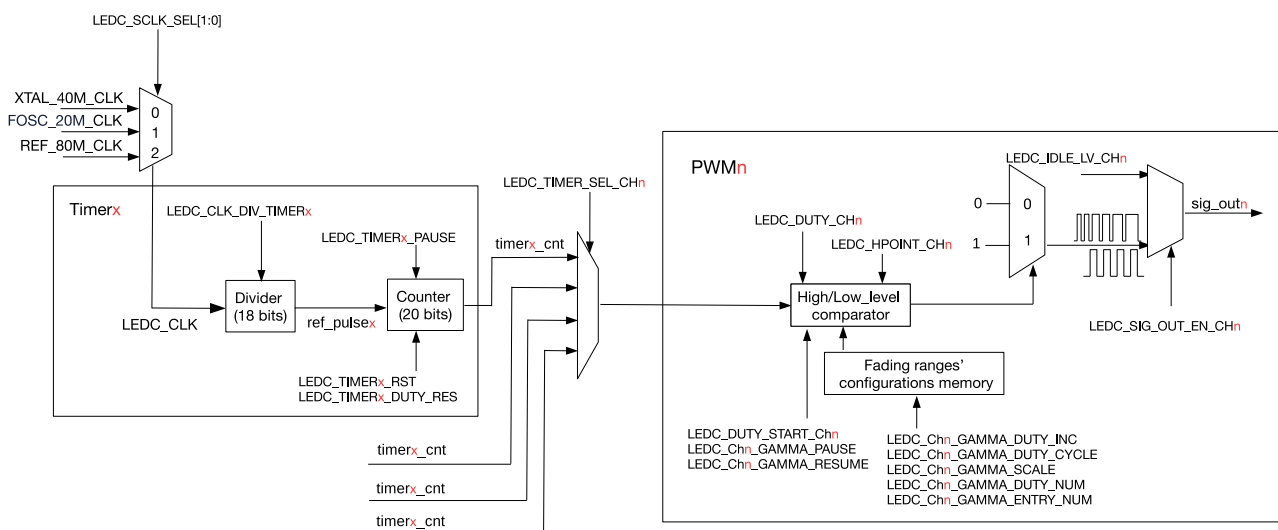


Figure 55.3-2. Timer and PWM Generator Block Diagram

55.4 Functional Description

55.4.1 Timers

Each timer in LED PWM Controller internally maintains a timebase counter. Referring to Figure 55.3-2, this clock signal used by the timebase counter is named `ref_pulsex`. All timers use the same clock source `LEDC_CLK`, which is then passed through a clock divider to generate `ref_pulsex` for the counter.

55.4.1.1 Clock Source

LED PWM registers configured by software are clocked by APB_CLK. To use the LED PWM peripheral, the APB_CLK signal going to the LED PWM has to be enabled. The APB_CLK signal to LED PWM can be enabled by setting the HP_SYS_CLKRST_LEDC_APB_CLK_EN field in the HP_SYS_CLKRST_SOC_CLK_CTRL3_REG register. The LEDC_CLK signal to LED PWM can be enabled by setting the HP_SYS_CLKRST_LEDC_CLK_EN field in the HP_SYS_CLKRST_PERI_CLK_CTRL22_REG register. The LED PWM peripheral can be reset via software by setting the HP_SYS_CLKRST_RST_EN_LEDC field in the HP_SYS_CLKRST_HP_RST_EN1_REG register.

Timers in the LED PWM Controller choose their common clock source from one of the following clock signals: XTAL_40M_CLK, FOSC_20M_CLK, and REF_80M_CLK. The procedure for selecting a clock source signal for LEDC_CLK is described below:

- XTAL_40M_CLK: Set PCR_LEDC_SCLK_SEL[1:0] to 0
- FOSC_20M_CLK: Set PCR_LEDC_SCLK_SEL[1:0] to 1
- REF_80M_CLK: Set PCR_LEDC_SCLK_SEL[1:0] to 2

The LEDC_CLK signal will then be passed through the clock divider.

For more information, please refer to Chapter 10 [Reset and Clock](#).

55.4.1.2 Clock Divider Configuration

The LEDC_CLK signal is passed through a clock divider to generate the ref_pulse_x signal for the counter. The frequency of ref_pulse_x is equal to the frequency of LEDC_CLK divided by the divisor LEDC_CLK_DIV (see Figure 55.3-2).

The clock divider is a fractional divider. Thus, the divisor LEDC_CLK_DIV can be non-integer values. LEDC_CLK_DIV is configured according to the following equation.

$$\text{LEDC_CLK_DIV} = A + \frac{B}{256}$$

- The integer part A corresponds to the most significant 10 bits of LEDC_CLK_DIV_TIMER_x (i.e., LEDC_TIMER_x_CONF_REG[22:13])
- The fractional part B corresponds to the least significant 8 bits of LEDC_CLK_DIV_TIMER_x (i.e., LEDC_TIMER_x_CONF_REG[12:5])

When the fractional part B is 0, LEDC_CLK_DIV is equivalent to an integer divisor (i.e., an integer prescaler). In other words, a ref_pulse_x clock pulse is generated after every A number of LEDC_CLK clock pulses.

However, when B is not 0, LEDC_CLK_DIV becomes a non-integer divisor. The clock divider implements non-integer frequency division by alternating between A and $(A+1)$ LEDC_CLK clock pulses per ref_pulse_x clock pulse. In this way, the average frequency of ref_pulse_x clock pulse will be the desired frequency (i.e., the non-integer divided frequency). For every 256 ref_pulse_x clock pulses:

- A number of B ref_pulse_x clock pulses are generated every $(A+1)$ LEDC_CLK clock pulses
- A number of $(256-B)$ ref_pulse_x clock pulses are generated every A LEDC_CLK clock pulses
- The ref_pulse_x clock pulses generated every $(A+1)$ pulses are evenly distributed amongst those generated every A pulses

Figure 55.4-1 illustrates the relation between LEDC_CLK clock pulses and ref_pulse x clock pulses when LEDC_CLK_DIV is a non-integer value.

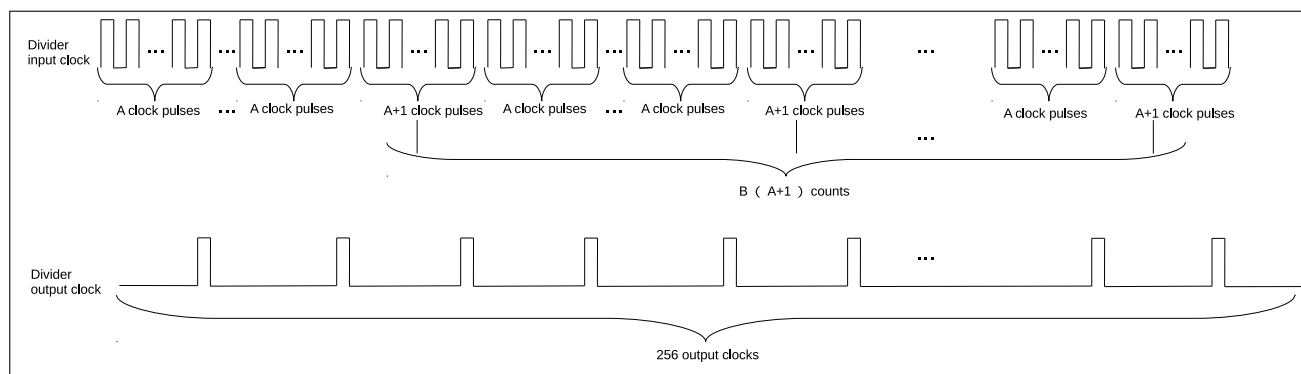


Figure 55.4-1. Frequency Division When LEDC_CLK_DIV is a Non-Integer Value

To change the timer's clock divisor at runtime, first configure the LEDC_CLK_DIV_TIMER x field, and then set the LEDC_TIMER x _PARA_UP field to apply the new configuration. This will cause the newly configured values to take effect upon the next overflow of the counter. The LEDC_TIMER x _PARA_UP field will be automatically cleared by hardware.

55.4.1.3 20-Bit Counter

Each timer contains a 20-bit timebase counter that uses ref_pulse x as its reference clock (see Figure 55.3-2). The LEDC_TIMER x _DUTY_RES field configures the actual used bit width of this 20-bit counter. Hence, the maximum resolution of the PWM signal is 20 bits. The counter counts up to $2^{\text{LEDC_TIMER}_x\text{_DUTY_RES}} - 1$, overflows and begins counting from 0 again. The counter's value can be read, reset, and suspended by software. Figure 55.4-2 shows the relationship between the counter and PWM resolution.

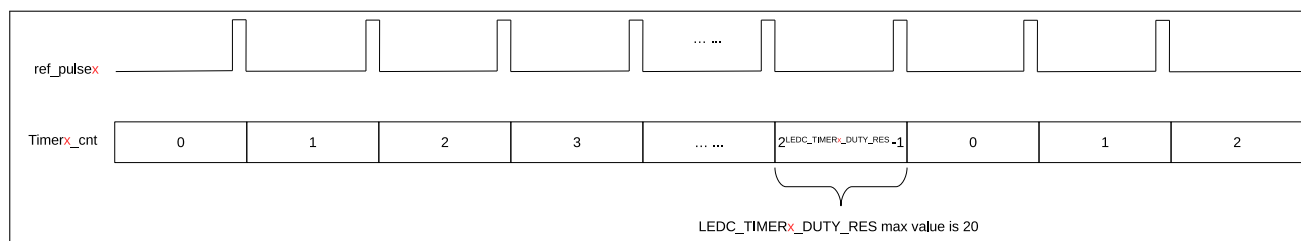


Figure 55.4-2. Relationship Between Counter And Resolution

Every time the counter overflows, it can trigger the LEDC_TIMER x _OVF_INT interrupt (generated automatically by hardware without configuration). It can also be configured to trigger LEDC_OVF_CNT_CH n _INT interrupt after overflowing LEDC_OVF_NUM_CH n + 1 times. To configure LEDC_OVF_CNT_CH n _INT interrupt, please:

1. Configure LEDC_TIMER_SEL_CH n to select the timer for the PWM generator
2. Enable the overflow counter by setting LEDC_OVF_CNT_EN_CH n
3. Configure LEDC_OVF_NUM_CH n with the number of counter overflows (that triggers an interrupt) minus 1
4. Enable the overflow interrupt by setting LEDC_OVF_CNT_CH n _INT_ENA
5. Configure LEDC_TIMER x _DUTY_RES to specify the counter bit width for the selected Timer x and wait for a LEDC_OVF_CNT_CH n _INT interrupt

To change the overflow value at runtime, first set the `LEDC_TIMERx_DUTY_RES` field, and then set the `LEDC_TIMERx_PARA_UP` field. This will cause the newly configured values to take effect upon the next overflow of the counter. If `LEDC_OVF_CNT_EN_CHn` field is reconfigured, `LEDC_PARA_UP_CHn` should be set to apply the new configuration. In summary, these configuration values need to be updated by setting `LEDC_TIMERx_PARA_UP` or `LEDC_PARA_UP_CHn`. `LEDC_TIMERx_PARA_UP` and `LEDC_PARA_UP_CHn` will be automatically cleared by hardware.

Referring to Figure 55.3-2, the frequency of a PWM generator output signal (`sig_outn`) is dependent on the frequency of the timer's clock source `LEDC_CLK`, the clock divisor `LEDC_CLK_DIV`, and the duty resolution (counter width) `LEDC_TIMERx_DUTY_RES`:

$$f_{\text{PWM}} = \frac{f_{\text{LEDC_CLK}}}{\text{LEDC_CLK_DIV} \cdot 2^{\text{LEDC_TIMERx_DUTY_RES}}}$$

Based on the formula above, the desired duty resolution can be calculated as follows:

$$\text{LEDC_TIMERx_DUTY_RES} = \log_2 \left(\frac{f_{\text{LEDC_CLK}}}{f_{\text{PWM}} \cdot \text{LEDC_CLK_DIV}} \right)$$

Table 55.4-1 lists the commonly-used frequencies and their corresponding resolutions.

Table 55.4-1. Commonly-used Frequencies and Resolutions

LEDC_CLK	PWM Frequency	Highest Resolution (bit) ¹	Lowest Resolution (bit) ²
REF_80M_CLK (80 MHz)	1 kHz	16	6
REF_80M_CLK (80 MHz)	5 kHz	13	3
REF_80M_CLK (80 MHz)	10 kHz	12	2
XTAL_40M_CLK (40 MHz)	1 kHz	15	5
XTAL_40M_CLK (40 MHz)	4 kHz	13	3
FOSC_20M_CLK (20 MHz)	1 kHz	14	4
FOSC_20M_CLK (20 MHz)	2 kHz	13	3

¹ The highest resolution is calculated when the clock divisor `LEDC_CLK_DIV` is 1. If the highest resolution calculated by the formula is higher than the counter's width 20 bits, then the highest resolution should be 20 bits.

² The lowest resolution is calculated when the clock divisor `LEDC_CLK_DIV` is $1023 + \frac{255}{256}$. If the lowest resolution calculated by the formula is lower than 0, then the lowest resolution should be 1.

55.4.2 PWM Generators

To generate a PWM signal, a PWM generator (`PWMn`) needs a timer (`Timerx`). Each PWM generator can be configured separately by setting `LEDC_TIMER_SEL_CHn` to use one of four timers to generate the PWM output.

As shown in Figure 55.3-2, each PWM generator has a comparator and two multiplexers. A PWM generator compares the timer's 20-bit counter value (`Timerx_cnt`) to two trigger values `Hpointn` and `Lpointn`. When the timer's counter value is equal to `Hpointn` or `Lpointn`, the PWM signal is high or low, respectively, as described below:

- If `Timer $x$ _cnt == Hpoint $n$` , `sig_out $n$` is 1.
- If `Timer $x$ _cnt == Lpoint $n$` , `sig_out $n$` is 0.

Figure 55.4-3 illustrates how `Hpoint $n$` and `Lpoint $n$` are used to generate a fixed duty cycle PWM output signal.

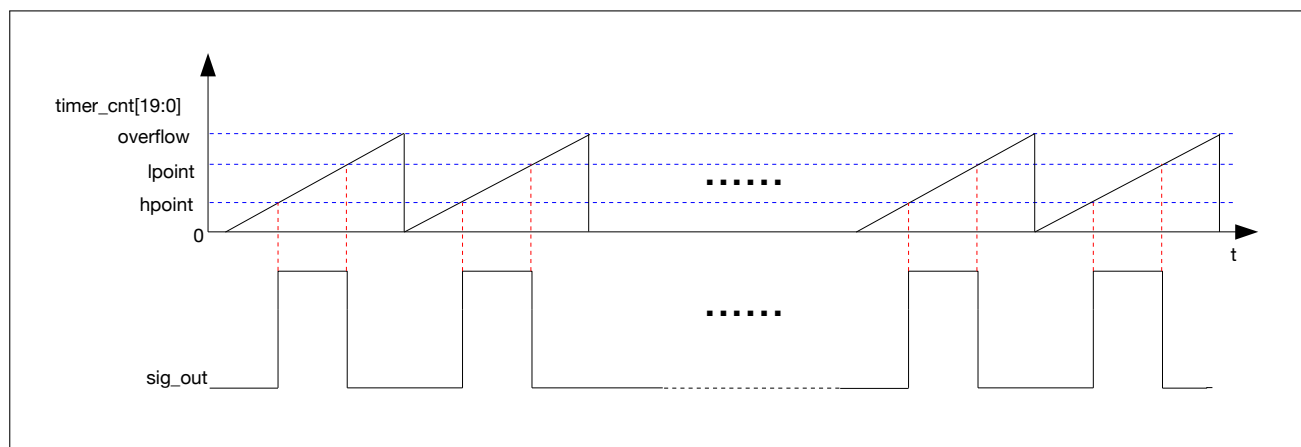


Figure 55.4-3. LED PWM Output Signal Diagram

For a particular PWM generator (PWM n), its `Hpoint $n$` is sampled from the `LEDC_HPOINT_CH $n$` field each time the selected timer's counter overflows. Likewise, `Lpoint $n$` is also sampled on every counter overflow and is calculated from the sum of the `LEDC_DUTY_CH $n$ [24:4]` and `LEDC_HPOINT_CH $n$` fields. By setting `Hpoint $n$` and `Lpoint $n$` via the `LEDC_HPOINT_CH $n$` and `LEDC_DUTY_CH $n$ [24:4]` fields, the relative phase and duty cycle of the PWM output can be set.

The PWM output signal (`sig_out $n$`) is enabled by setting `LEDC_SIG_OUT_EN_CH $n$` . When `LEDC_SIG_OUT_EN_CH $n$` is cleared, PWM signal output is disabled, and the output signal (`sig_out $n$`) will output a constant level specified by `LEDC_IDLE_LV_CH $n$` .

The bits `LEDC_DUTY_CH $n$ [3:0]` are used to dither the duty cycles of the PWM output signal (`sig_out $n$`) by periodically altering the duty cycle of `sig_out $n$` . When `LEDC_DUTY_CH $n$ [3:0]` is not 0, then for every 16 cycles of `sig_out $n$` , `LEDC_DUTY_CH $n$ [3:0]` of those cycles will have PWM pulses that are one timer tick longer than the other (16- `LEDC_DUTY_CH $n$ [3:0]`) cycles. For instance, if `LEDC_DUTY_CH $n$ [24:4]` is set to 10 and `LEDC_DUTY_CH $n$ [3:0]` is set to 5, then 5 of 16 cycles will have a PWM pulse with a duty value of 11 and the rest of the 16 cycles will have a PWM pulse with a duty value of 10. The average duty cycle after 16 cycles is 10.3125.

If fields `LEDC_TIMER_SEL_CH $n$` , `LEDC_HPOINT_CH $n$` , `LEDC_DUTY_CH $n$ [24:4]`, and `LEDC_SIG_OUT_EN_CH $n$` are reconfigured, `LEDC_PARA_UP_CH $n$` must be set to apply the new configuration. This will cause the newly configured values to take effect upon the next overflow of the counter. `LEDC_PARA_UP_CH $n$` field will be automatically cleared by hardware.

55.4.3 Duty Cycle Fading

The PWM generators can fade the duty cycle of a PWM output signal (i.e., gradually change the duty cycle from one value to another). Each PWM generator can have up to 16 duty cycle ranges, which can be independently configured in terms of fading direction (increase or decrease), fading amount, the number of fades, and fading frequency. If Duty Cycle Fading is enabled, every range's `Lpoint $n$` value will change

according to its fading configuration.

55.4.3.1 Linear Duty Cycle Fading

Linear fading PWM signals can be generated by configuring the direction, fading amount, the number of fades, and fading frequency of the first duty cycle range.

The programming procedures will be described in detail in the section [55.6](#).

After the procedures, the PWM generator can fade the duty cycle of a PWM signal once per LEDC_CH n _GAMMA_DUTY_CYCLE times of counter overflows. Every time when the PWM signal is faded, Lpoint n increases or decreases (configured by LEDC_CH n _GAMMA_DUTY_INC) by LEDC_CH n _GAMMA_SCALE, and the duty cycle increases or decreases (configured by LEDC_CH n _GAMMA_DUTY_INC) by

$$\frac{\text{LEDC_CH}_n\text{_GAMMA_SCALE}}{\text{LEDC_TIMER}_x\text{_DUTY_RES}}$$

The duty cycle is faded for LEDC_CH n _GAMMA_DUTY_NUM times. After that, the PWM generator stops fading and keeps outputting signals at this duty cycle. Upon each fading the duty cycle increases or decreases by the same amount, and therefore the PWM signal is a linear fading signal.

Figure [55.4-4](#) shows a linear fading PWM signal.

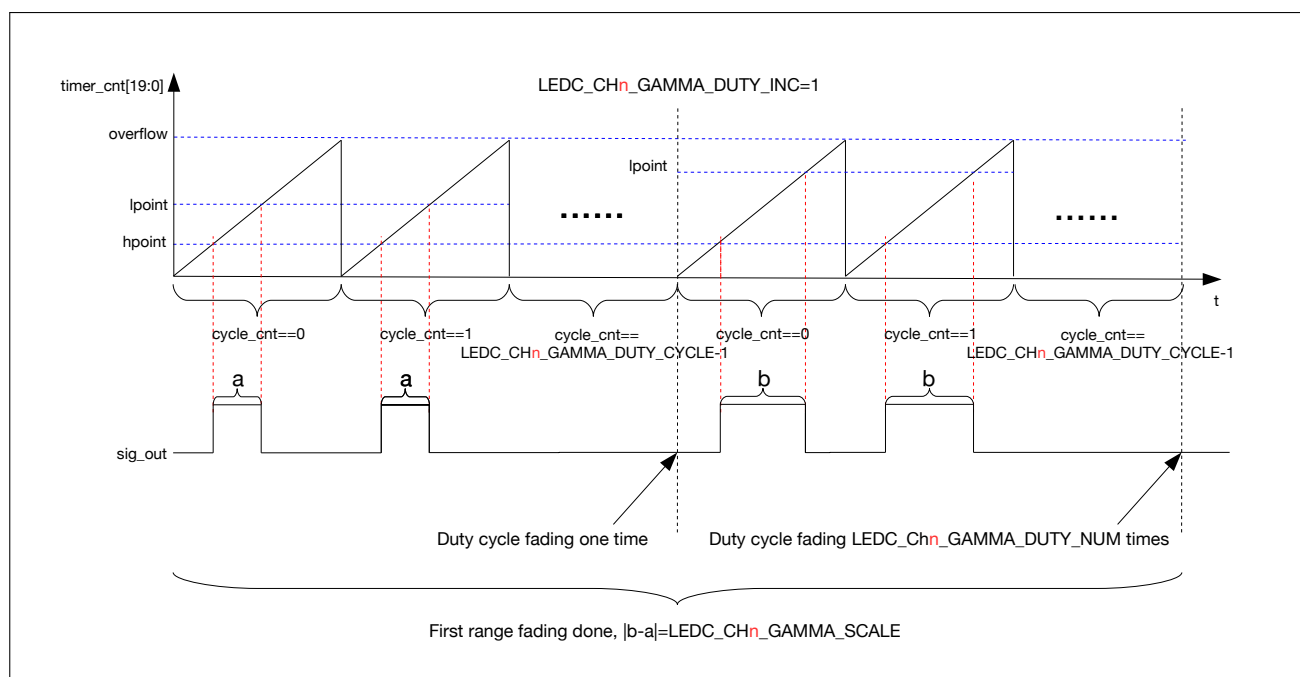


Figure 55.4-4. Output Signal of Linear Duty Cycle Fading

55.4.3.2 Gamma Curve Fading

Gamma curve fading PWM signals can be generated by configuring the fading direction, fading amount, the number of fades and fading frequency of multiple duty cycle fading ranges.

The programming procedures will be described in detail in the section [55.6](#).

After the procedures, the PWM generator can generate a PWM signal with `LEDC_CH $n$ _GAMMA_ENTRY_NUM` ranges. The duty cycle of the PWM signal fades according to the configurations of range 0 first, and then range 1, till range (`LEDC_CH $n$ _GAMMA_ENTRY_NUM` – 1) (the last range) where Duty Cycle Fading ends. The PWM signal fades independently in each range. According to the configuration of each range, every time when the counter overflows for `LEDC_CH $n$ _GAMMA_DUTY_CYCLE` times, `Lpoint $n$` increases or decreases (configured by `LEDC_CH $n$ _GAMMA_DUTY_INC`) by `LEDC_CH $n$ _GAMMA_SCALE`, and accordingly the duty cycle increases or decreases (configured by `LEDC_CH $n$ _GAMMA_DUTY_INC`) by

$$\frac{\text{LEDC_CH}_n\text{_GAMMA_SCALE}}{\text{LEDC_TIMER}_x\text{_DUTY_RES}}$$

After the duty cycle fades for `LEDC_CH $n$ _GAMMA_DUTY_NUM` times in a range, Duty Cycle Fading in this range finishes.

When Duty Cycle Fading finishes in all ranges (the number of ranges is specified by `LEDC_CH $n$ _GAMMA_ENTRY_NUM`), the PWM signal stops fading and keeps the duty cycle of the last fade. Given that the duty cycle fades differently and linearly in each range, several linear fading ranges would be fitted to a gamma curve.

Figure 55.4-5 illustrates a gamma curve fading PWM signal.

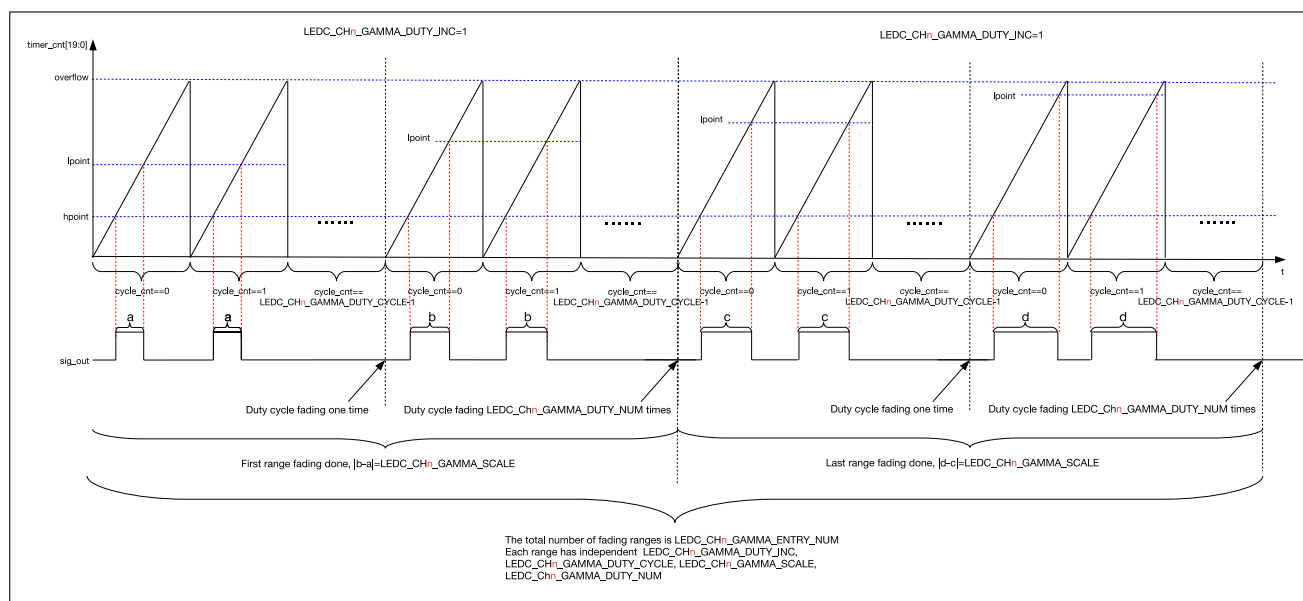


Figure 55.4-5. Output Signal of Gamma Curve Fading

55.4.3.3 Suspend and Resume Duty Cycle Fading

To suspend Duty Cycle Fading that has already been started, write 1 to the `LEDC_CH $n$ _GAMMA_PAUSE` field of the `LEDC_CH $n$ _GAMMA_CONF_REG` register. Once `LEDC_CH $n$ _GAMMA_PAUSE` is set to 1, the PWM signal keeps the duty cycle of the most recent fade.

To resume Duty Cycle Fading, write 1 to the `LEDC_CH $n$ _GAMMA_RESUME` field of the `LEDC_CH $n$ _GAMMA_CONF_REG` register. Once `LEDC_CH $n$ _GAMMA_RESUME` is set to 1, the PWM signal resumes fading from the range where the suspension occurs, until fading in the last range finishes. The fading will continue from the state when it was paused until all the ranges complete duty cycle fading (when

`LEDC_CH $n$ _GAMMA_RESUME` is set to 1, `LEDC_CH $n$ _GAMMA_PAUSE` is cleared automatically by hardware.

55.4.4 Event Task Matrix Feature

The LEDC on ESP32-P4 supports the Event Task Matrix (ETM) function, which allows LEDC's ETM tasks to be triggered by any peripherals' ETM events, or LEDC's ETM events to trigger any peripherals' ETM tasks. This section introduces the ETM tasks and events related to LEDC. For more information, please refer to [Chapter 13 Event Task Matrix \(ETM\)](#).

ETM-related events and tasks are enabled by configuring corresponding fields of `LEDC_EVT_TASK_ENO_REG`, `LEDC_EVT_TASK_EN1_REG` and `LEDC_EVT_TASK_EN2_REG` registers. For the correspondence between events, tasks, and fields, Please refer to [Section 55.8](#)).

LEDC can receive the following ETM tasks:

- `LEDC_TASK_DUTY_SCALE_UPDATE_CH $n$` : If the `LEDC_TASK_DUTY_SCALE_UPDATE_CH $n$ _EN` field is enabled, upon receiving the `LEDC_TASK_DUTY_SCALE_UPDATE_CH $n$` task, PWM n generates fading PWM signals according to the newly configured `LEDC_CH $n$ _GAMMA_SCALE` field.
- `LEDC_TASK_TIMER $x$ _RES_UPDATE`: If the `LEDC_TASK_TIMER $x$ _RES_UPDATE_EN` field is enabled, upon receiving the `LEDC_TASK_TIMER $x$ _RES_UPDATE` task, Timer x updates its counter's overflow value to the value configured in the `LEDC_TIMER $x$ _DUTY_RES` field at the next overflow of the counter.
- `LEDC_TASK_TIMER $x$ _CAP`: If the `LEDC_TASK_TIMER $x$ _CAP_EN` field is enabled, upon receiving the `LEDC_TASK_TIMER $x$ _CAP` task, Timer x captures its counter's value, and stores the value into the `LEDC_TIMER $x$ _CNT_CAP` field of register `LEDC_TIMER $x$ _CNT_CAP_REG`.
- `LEDC_TASK_SIG_OUT_DIS_CH $n$` : If the `LEDC_TASK_SIG_OUT_DIS_CH $n$ _EN` field is enabled, upon receiving the `LEDC_TASK_SIG_OUT_DIS_CH $n$` task, PWM n 's signal output is disabled, and the output signal (sig_out n) outputs a constant level as specified by field `LEDC_IDLE_LV_CH $n$` , as shown in [Figure 55.3-2](#).
- `LEDC_TASK_OVF_CNT_RST_CH $n$` : If the `LEDC_TASK_OVF_CNT_RST_CH $n$ _EN` field is enabled, upon receiving the `LEDC_TASK_OVF_CNT_RST_CH $n$` task, PWM n timer's overflow counter is reset to 0.
- `LEDC_TASK_TIMER $x$ _RST`: If the `LEDC_TASK_TIMER $x$ _RST_EN` field is enabled, upon receiving the `LEDC_TASK_TIMER $x$ _RST` task, Timer x 's counter is reset to 0.
- `LEDC_TASK_TIMER $x$ _RESUME` and `LEDC_TASK_TIMER $x$ _PAUSE`: If the `LEDC_TASK_TIMER $x$ _PAUSE_RESUME_EN` field is enabled, upon receiving the `LEDC_TASK_TIMER $x$ _RESUME` and `LEDC_TASK_TIMER $x$ _PAUSE` task, Timer x is suspended and resumed alternately. That is, when the task is received, Timer x is paused; and when the task is received again, Timer x is resumed.
- `LEDC_TASK_GAMMA_RESTART_CH $n$` : If the `LEDC_TASK_GAMMA_RESTART_CH $n$ _EN` field is enabled, upon receiving the `LEDC_TASK_GAMMA_RESTART_CH $n$` task, the PWM n restarts to generate the fading PWM signal.
- `LEDC_TASK_GAMMA_PAUSE_CH $n$` : If the `LEDC_TASK_GAMMA_PAUSE_CH $n$ _EN` field is enabled, upon receiving the `LEDC_TASK_GAMMA_PAUSE_CH $n$` task, PWM n suspends Duty Cycle Fading at the next timer overflow. That is, after the task has been received, PWM n keeps the duty cycle of the last fade.

- LEDC_TASK_GAMMA_RESUME_CH n : If the [LEDC_TASK_GAMMA_RESUME_CH \$n\$ _EN](#) field is enabled, upon receiving the LEDC_TASK_GAMMA_RESUME_CH n task, PWM n resumes Duty Cycle Fading at the next counter overflow. That is, after the task has been received, PWM n resumes fading from the range where the suspension occurs.

LEDC can generate the following ETM events:

- LEDC_EVT_DUTY_CHNG_END_CH n : Generated when the [LEDC_EVT_DUTY_CHNG_END_CH \$n\$ _EN](#) field is enabled, and PWM n has finished Duty Cycle Fading.
- LEDC_EVT_OVF_CNT_PLS_CH n : Generated when the [LEDC_EVT_OVF_CNT_PLS_CH \$n\$ _EN](#) field is enabled and when PWM n timer's counter overflows for [LEDC_OVF_NUM_CH \$n\$](#) + 1 times.
- LEDC_EVT_TIME_OVF_TIMER x : Generated when the [LEDC_EVT_TIME_OVF_TIMER \$x\$ _EN](#) field is enabled and Timer x 's counter overflows.
- LEDC_EVT_TIMER x _CMP: Generated when the [LEDC_EVT_TIMER \$x\$ _CMP_EN](#) field is enabled and the value of Timer x 's counter reaches that of the [LEDC_TIMER \$x\$ _CMP](#) field of register [LEDC_TIMER \$x\$ _CMP_REG](#).

In practical applications, LEDC's ETM events can trigger its own ETM tasks. For example, LEDC_EVT_DUTY_CHNG_END_CH n event can trigger the LEDC_TASK_GAMMA_RESTART_CH n task, thus starting the next fading directly after the current fading is completed.

55.5 Interrupts

ESP32-P4's LEDC can generate the LEDC_INT interrupt signal, which will be sent to the [Interrupt Matrix](#).

Interrupt signal LEDC_INT can be generated by the following internal interrupt sources:

- LEDC_OVF_CNT_CH n _INT: Triggered when the timer counter overflows for [LEDC_OVF_NUM_CH \$n\$](#) + 1 times and the register [LEDC_OVF_CNT_EN_CH \$n\$](#) is set to 1.
- LEDC_DUTY_CHNG_END_CH n _INT: Triggered when a fade on PWM n has finished.
- LEDC_TIMER x _OVF_INT: Triggered when Timer x has reached its maximum counter value.

The above interrupt sources can only be triggered when the interrupt enable register [XXX_ENA](#) is set to 1. Write 1 to [XXX_CLR](#) will clear the interrupt ([XXX](#) is the interrupt source name).

55.6 Programming Procedures

The configuration process of the LED PWM controller to generate PWM signals is as follows:

1. Configure the Timer x according to section [55.4.1](#). After that, the clock source, divider, and counter are ready.
2. Configure the PWM n generator according to section [55.4.2](#). After that, PWM n selects one of the four timers and the phase and duty cycle of the PWM n output is set.
3. Choose the corresponding configuration process based on the different duty cycle fading types:
 - (a) Linear duty cycle fading:
 - i. Configure the [LEDC_DUTY_CH \$n\$](#) field with the initial value of [Lpoint \$n\$](#) .

- ii. Set the [LEDC_DUTY_START_CH \$n\$](#) field to enable Duty Cycle Fading. When this field is cleared, Duty Cycle Fading will be disabled.
 - iii. Write the linear duty cycle fading configuration to (LEDC_CH n _MEM starting address + duty cycle range number * 4), the LEDC_CH n _MEM starting address is provided in Table 55.7-1. Please note that the following bit names are just for easier description, and there are no such register fields.
 - Bit 0 of the configuration data is used to configure the direction (hereafter referred to as LEDC_CH n _GAMMA_DUTY_INC). When it is set or cleared, the Lpoint n will increase or decrease in the current configured range.
 - Bit 1 to 10 of the configuration data is used to configure the number of times the counter overflows per an increment or decrement of Lpoint n (hereafter referred to as LEDC_CH n _GAMMA_DUTY_CYCLE). In other words, Lpoint n will increase or decrease after the counter overflows for the configured number of times.
 - Bit 11 to 20 of the configuration data is used to configure the amount by which Lpoint n increase or decrease in the current configured range (hereafter referred to as LEDC_CH n _GAMMA_SCALE).
 - Bit 21 to 30 of the configuration data is used to configure the number of fades in the current configured range (hereafter referred to as LEDC_CH n _GAMMA_DUTY_NUM).
 - The duty cycle range number (from 0 to 15) specifies to which range the above configuration data apply. For linear duty cycle fading only the first range needs to be configured, so the duty cycle range number should be configured as 0.
 - iv. Configure the number of ranges per each fading (1 in this case) via the [LEDC_CH \$n\$ _GAMMA_ENTRY_NUM](#) field of the [LEDC_CH \$n\$ _GAMMA_CONF_REG](#). Once the specified number of ranges have been faded, Duty Cycle Fading stops and the PWM generator triggers the [LEDC_DUTY_CHNG_END_CH \$n\$ _INT](#) interrupt. For linear duty cycle fading there is only one duty cycle range (i.e., the first one), so configure [LEDC_CH \$n\$ _GAMMA_ENTRY_NUM](#) as 1.
 - v. Set the [LEDC_PARA_UP_CH \$n\$](#) field to apply the above configurations. After this field is set, the configurations for Duty Cycle Fading will take effect upon the next overflow of the counter, and the PWM generator will output a linear fading PWM signal following configurations. [LEDC_PARA_UP_CH \$n\$](#) field will be automatically cleared by hardware.
- (b) Gamma curve fading:
- i. The same as Step 1 in Section 55.4.3.1.
 - ii. The same as Step 2 in Section 55.4.3.1.
 - iii. Configure multiple duty cycle ranges with the following steps. Write the gamma curve fading configuration to (LEDC_CH n _MEM starting address + duty cycle range number * 4), the LEDC_CH n _MEM starting address is provided in Table 55.7-1.
 - Bit 0 of the configuration data is used to configure the direction (hereafter referred to as LEDC_CH n _GAMMA_DUTY_INC). When it is set or cleared, the Lpoint n will increment or decrement in the current configured range.

- Bit 1 to 10 of the configuration data is used to configure the number of times the counter overflows per an increase or decrease of Lpoint n (hereafter referred to as LEDC_CH n _GAMMA_DUTY_CYCLE). In other words, Lpoint n will increase or decrease after the counter overflows for the configured number of times.
 - Bit 11 to 20 of the configuration data is used to configure the amount by which Lpoint n increase or decrease in the current configured range (hereafter referred to as LEDC_CH n _GAMMA_SCALE).
 - Bit 21 to 30 of the configuration data is used to configure the number of fades in the current configured range (hereafter referred to as LEDC_CH n _GAMMA_DUTY_NUM).
 - The duty cycle range number (from 0 to 15) specifies to which range the above configuration data apply. For gamma curve fading, it must start from 0 and increase by 1 for the next range to be configured.
 - Once the above procedures are finished, the configuration for one range is complete. Other ranges are configured by repeating the same set of procedures. You can configure any number of ranges from 0 to 15, and each can be configured independently.
- iv. After all required ranges are configured, write the total number of ranges configured in Step 3 to the LEDC_CH n _GAMMA_ENTRY_NUM field of the LEDC_CH n _GAMMA_CONF_REG register.
- v. Set the LEDC_PARA_UP_CH n field to apply the above configuration. After this field is set, the configurations for duty cycle fading will take effect upon the next overflow of the counter, and the PWM generator will output a gamma curve fading PWM signal following the configurations. LEDC_PARA_UP_CH n field will be automatically cleared by hardware.

After the above procedures, the LED PWM controller will generate the desired PWM signals.

At any time, duty cycle fading can be suspended or resumed, more details can be found in section 55.4.3.3. If the duty cycle fading configurations need to be changed before the PWM signals stop fading (that is, the LEDC_DUTY_CHNG_END_CH n _INT has not been triggered), the duty cycle fading needs to be suspended before changing the configurations. However, if the duty cycle fading configurations need to be changed after the PWM signals stop fading (that is, the LEDC_DUTY_CHNG_END_CH n _INT has been triggered), the configurations can be changed directly.

At any time, PWM signal output can be enabled or disabled by software, and the output signal level can be changed, more details can be found in section 55.4.2.

55.7 Memory Blocks

Each LEDC channel has a memory to store duty cycle fading configuration data. Each memory can store configuration data for up to 16 ranges. The addresses in this section are relative to LED PWM Controller base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

Table 55.7-1. LEDC Memory Blocks

Name	Description	Size	Starting Address	Ending Address	Access
LEDC_CH n _MEM	Memory of PWM n	64 B	$0x400 + \text{PWM}n * 64$	$0x400 + (\text{PWM}n + 1) * 64$	WR

55.8 Register Summary

The addresses in this section are relative to LED PWM Controller base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

The abbreviations given in Column **Access** are explained in Section *Access Types for Registers*.

Name	Description	Address	Access
Configuration Register			
LEDC_CHO_CONFO_REG	Configuration register 0 for channel 0	0x0000	varies
LEDC_CHO_HPOINT_REG	High point register for channel 0	0x0004	R/W
LEDC_CHO_DUTY_REG	Initial duty cycle register for channel 0	0x0008	R/W
LEDC_CHO_CONF1_REG	Configuration register 1 for channel 0	0x000C	R/W/SC
LEDC_CH1_CONFO_REG	Configuration register 0 for channel 1	0x0014	varies
LEDC_CH1_HPOINT_REG	High point register for channel 1	0x0018	R/W
LEDC_CH1_DUTY_REG	Initial duty cycle register for channel 1	0x001C	R/W
LEDC_CH1_CONF1_REG	Configuration register 1 for channel 1	0x0020	R/W/SC
LEDC_CH2_CONFO_REG	Configuration register 0 for channel 2	0x0028	varies
LEDC_CH2_HPOINT_REG	High point register for channel 2	0x002C	R/W
LEDC_CH2_DUTY_REG	Initial duty cycle register for channel 2	0x0030	R/W
LEDC_CH2_CONF1_REG	Configuration register 1 for channel 2	0x0034	R/W/SC
LEDC_CH3_CONFO_REG	Configuration register 0 for channel 3	0x003C	varies
LEDC_CH3_HPOINT_REG	High point register for channel 3	0x0040	R/W
LEDC_CH3_DUTY_REG	Initial duty cycle register for channel 3	0x0044	R/W
LEDC_CH3_CONF1_REG	Configuration register 1 for channel 3	0x0048	R/W/SC
LEDC_CH4_CONFO_REG	Configuration register 0 for channel 4	0x0050	varies
LEDC_CH4_HPOINT_REG	High point register for channel 4	0x0054	R/W
LEDC_CH4_DUTY_REG	Initial duty cycle register for channel 4	0x0058	R/W
LEDC_CH4_CONF1_REG	Configuration register 1 for channel 4	0x005C	R/W/SC
LEDC_CH5_CONFO_REG	Configuration register 0 for channel 5	0x0064	varies
LEDC_CH5_HPOINT_REG	High point register for channel 5	0x0068	R/W
LEDC_CH5_DUTY_REG	Initial duty cycle register for channel 5	0x006C	R/W
LEDC_CH5_CONF1_REG	Configuration register 1 for channel 5	0x0070	R/W/SC
LEDC_CH6_CONFO_REG	Configuration register 0 for channel 6	0x0078	varies
LEDC_CH6_HPOINT_REG	High point register for channel 6	0x007C	R/W
LEDC_CH6_DUTY_REG	Initial duty cycle register for channel 6	0x0080	R/W
LEDC_CH6_CONF1_REG	Configuration register 1 for channel 6	0x0084	R/W/SC
LEDC_CH7_CONFO_REG	Configuration register 0 for channel 7	0x008C	varies
LEDC_CH7_HPOINT_REG	High point register for channel 7	0x0090	R/W
LEDC_CH7_DUTY_REG	Initial duty cycle register for channel 7	0x0094	R/W
LEDC_CH7_CONF1_REG	Configuration register 1 for channel 7	0x0098	R/W/SC
LEDC_TIMER0_CONF_REG	Timer 0 configuration register	0x00A0	varies
LEDC_TIMER1_CONF_REG	Timer 1 configuration register	0x00A8	varies
LEDC_TIMER2_CONF_REG	Timer 2 configuration register	0x00B0	varies
LEDC_TIMER3_CONF_REG	Timer 3 configuration register	0x00B8	varies

Name	Description	Address	Access
LEDC_CHO_GAMMA_CONF_REG	LEDC channel 0 gamma config register	0x0100	varies
LEDC_CH1_GAMMA_CONF_REG	LEDC channel 1 gamma config register	0x0104	varies
LEDC_CH2_GAMMA_CONF_REG	LEDC channel 2 gamma config register	0x0108	varies
LEDC_CH3_GAMMA_CONF_REG	LEDC channel 3 gamma config register	0x010C	varies
LEDC_CH4_GAMMA_CONF_REG	LEDC channel 4 gamma config register	0x0110	varies
LEDC_CH5_GAMMA_CONF_REG	LEDC channel 5 gamma config register	0x0114	varies
LEDC_CH6_GAMMA_CONF_REG	LEDC channel 6 gamma config register	0x0118	varies
LEDC_CH7_GAMMA_CONF_REG	LEDC channel 7 gamma config register	0x011C	varies
LEDC_EVT_TASK_ENO_REG	LEDC event task enable bit register 0	0x0120	R/W
LEDC_EVT_TASK_EN1_REG	LEDC event task enable bit register 1	0x0124	R/W
LEDC_EVT_TASK_EN2_REG	LEDC event task enable bit register 2	0x0128	R/W
LEDC_TIMER0_CMP_REG	LEDC timer 0 compare value register	0x0140	R/W
LEDC_TIMER1_CMP_REG	LEDC timer 1 compare value register	0x0144	R/W
LEDC_TIMER2_CMP_REG	LEDC timer 2 compare value register	0x0148	R/W
LEDC_TIMER3_CMP_REG	LEDC timer 3 compare value register	0x014C	R/W
LEDC_CONF_REG	LEDC global configuration register	0x0170	R/W
Status Register			
LEDC_CHO_DUTY_R_REG	Current duty cycle register for channel 0	0x0010	RO
LEDC_CH1_DUTY_R_REG	Current duty cycle register for channel 1	0x0024	RO
LEDC_CH2_DUTY_R_REG	Current duty cycle register for channel 2	0x0038	RO
LEDC_CH3_DUTY_R_REG	Current duty cycle register for channel 3	0x004C	RO
LEDC_CH4_DUTY_R_REG	Current duty cycle register for channel 4	0x0060	RO
LEDC_CH5_DUTY_R_REG	Current duty cycle register for channel 5	0x0074	RO
LEDC_CH6_DUTY_R_REG	Current duty cycle register for channel 6	0x0088	RO
LEDC_CH7_DUTY_R_REG	Current duty cycle register for channel 7	0x009C	RO
LEDC_TIMER0_VALUE_REG	Timer 0 current counter value register	0x00A4	RO
LEDC_TIMER1_VALUE_REG	Timer 1 current counter value register	0x00AC	RO
LEDC_TIMER2_VALUE_REG	Timer 2 current counter value register	0x00B4	RO
LEDC_TIMER3_VALUE_REG	Timer 3 current counter value register	0x00BC	RO
LEDC_TIMER0_CNT_CAP_REG	LEDC timer 0 captured count value register	0x0150	RO
LEDC_TIMER1_CNT_CAP_REG	LEDC timer 1 captured count value register	0x0154	RO
LEDC_TIMER2_CNT_CAP_REG	LEDC timer 2 captured count value register	0x0158	RO
LEDC_TIMER3_CNT_CAP_REG	LEDC timer 3 captured count value register	0x015C	RO
Interrupt Register			
LEDC_INT_RAW_REG	Interrupt raw status register	0x00C0	R/WTC/SS
LEDC_INT_ST_REG	Interrupt masked status register	0x00C4	RO
LEDC_INT_ENA_REG	Interrupt enable register	0x00C8	R/W
LEDC_INT_CLR_REG	Interrupt clear register	0x00CC	WT
Version Register			
LEDC_DATE_REG	Version control register	0x0174	R/W

55.9 Registers

The addresses in this section are relative to LED PWM Controller base address provided in Table 7.3-2 in Chapter 7 *System and Memory*.

For how to program reserved fields, please refer to Section [Programming Reserved Register Field](#).

Register 55.1. LEDC_CH n _CONFO_REG (n : 0-7) (0x0000+0x14* n)

(reserved)																LEDC_OVF_CNT_RESET_CH ⁿ LEDC_OVF_CNT_EN_CH ⁿ								LEDC_OVF_NUM_CH ⁿ				LEDC_PARA_UP_CH ⁿ LEDC_IDLE_LV_CH ⁿ LEDC_SIG_OUT_EN_CH ⁿ LEDC_TIMER_SEL_CH ⁿ			
31													17	16	15	14					5	4	3	2	1	0					
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0				0	0	0	0	0	0	Reset				

LEDC_TIMER_SEL_CH n Configures which timer is channel n selected.

- 0: Select Timer 0
 - 1: Select Timer 1
 - 2: Select Timer 2
 - 3: Select Timer 3
- (R/W)

LEDC_SIG_OUT_EN_CH n Configures whether to enable signal output on channel n .

- 0: Signal output disable
 - 1: Signal output enable
- (R/W)

LEDC_IDLE_LV_CH n Configures the output value when channel n is inactive. Valid only when [LEDC_SIG_OUT_EN_CH \$n\$](#) is 0.

- 0: Output level is low
 - 1: Output level is high
- (R/W)

LEDC_PARA_UP_CH n Configures whether to update [LEDC_HPOINT_CH \$n\$](#) , [LEDC_DUTY_START_CH \$n\$](#) , [LEDC_SIG_OUT_EN_CH \$n\$](#) , [LEDC_TIMER_SEL_CH \$n\$](#) , [LEDC_OVF_CNT_EN_CH \$n\$](#) fields and duty cycle range configurations for channel n , and will be automatically cleared by hardware.

- 0: Invalid. No effect
 - 1: Update
- (WT)

LEDC_OVF_NUM_CH n Configures the maximum times of overflow minus 1. The LEDC_OVF_CNT_CH n _INT interrupt will be triggered when channel n overflows for (LEDC_OVF_NUM_CH n + 1) times. (R/W)

LEDC_OVF_CNT_EN_CH n Configures whether to enable the ovf_cnt of channel n .

- 0: Disable
 - 1: Enable
- (R/W)

Continued on the next page...

Register 55.1. LEDC_CH n _CONFO_REG (n : 0-7) (0x0000+0x14* n)

Continued from the previous page...

LEDC_OVF_CNT_RESET_CH n Configures whether to reset the ovf_cnt of channel n .

0: Invalid. No effect

1: Reset the ovf_cnt

(WT)

Register 55.2. LEDC_CH*n*_HPOINT_REG (*n*: 0-7) (0x0004+0x14n*)**

Diagram illustrating the structure of the `LEDC_HPOINT_CHn` register:

- Bits 31 to 20 are reserved (indicated by a diagonal line and the text "(reserved)").
- Bits 19 to 0 contain the `LEDC_HPOINT_CHn` value, which is shown as `0x000`.
- The register is labeled "Reset" at the bottom right.

LEDC_HPOINT_CH n Configures high point of signal output on channel n . The output value changes to high when the selected timers have reached the value specified by this register. (R/W)

Register 55.3. LEDC_CH*n*_DUTY_REG (*n*: 0-7) (0x0008+0x14n*)**

Register structure for LEDC_DUTY_CHn:

31	25	24	0
(reserved)		LEDC_DUTY_CHn	
0 0 0 0 0 0 0		0x00000	

LEDC_DUTY_CH n Configures the duty cycle of signal output on channel n . (R/W)

Register 55.4. LEDC_CH n _CONF1_REG (n : 0-7) (0x000C+0x14* n)

Diagram illustrating the structure of the `LEDC_DUTY_START_CHn` register:

- Bit 31: `LEDC_DUTY_START_CHn`
- Bits 30-0: `(reserved)`
- Bit 0: `Reset`

LEDC_DUTY_START_CH_n Configures whether the duty cycle fading configurations take effect.

0: Not take effect

1: Take effect

(R/W/SC)

Register 55.5. LEDC_TIMER_x_CONF_REG (x: 0-3) (0x00A0+0x8*x)

(reserved)										LEDC_TIMER X _PARA_UP										(reserved)										LEDC_TIMER X _RST										LEDC_TIMER X _PAUSE																				LEDC_CLK_DIV_TIMER X																				LEDC_TIMER X _DUTY_RES									
31					27					26	25	24	23	22																		5					4	0																																																			
0					0					0	0	1	0	0x000																								0x0																		Reset																																	

LEDC_TIMERx_DUTY_RES Configures the bit width of the counter in timer **x**. Valid values are 1 to 20. (R/W)

LEDC_CLK_DIV_TIMER x Configures the divisor for the divider in timer x .

The least significant eight bits represent the fractional part. The most significant ten bits represent the integer part. (R/W)

LEDC_TIMER x _PAUSE Configures whether to pause the counter in timer x .

0: Normal

1: Pause

(R/W)

LEDC_TIMER_x_RST Configures whether to reset timer *x*. The counter will show 0 after reset.

0: Not reset

1: Reset

(R/W)

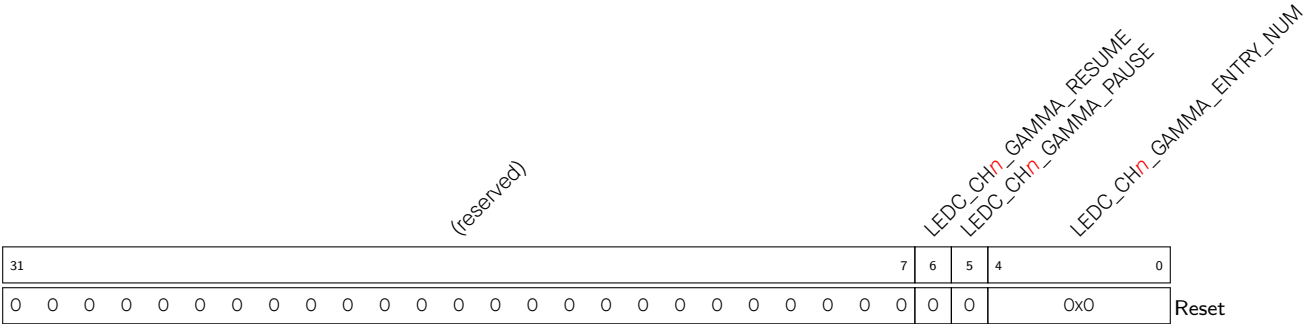
LEDC_TIMERx_PARA_UP Configures whether to update LEDC_CLK_DIV_TIMERx and LEDC_TIMERx DUTY RES.

0: Invalid. No effect

1: Update

(WT)

Register 55.6. LEDC_CHn_GAMMA_CONF_REG (n: 0-7) (0x0100+0x4*n)



LEDC_CHn_GAMMA_ENTRY_NUM Configures the number of duty cycle fading ranges for LEDC channel n. (R/W)

LEDC_CHn_GAMMA_PAUSE Configures whether to pause duty cycle fading of LEDC channel n.

0: Invalid. No effect

1: Pause

(WT)

LEDC_CHn_GAMMA_RESUME Configures whether to resume duty cycle fading of LEDC channel n.

0: Invalid. No effect

1: Resume

(WT)

Register 55.7. LEDC_EVT_TASK_EN0_REG (0x0120)

[illegible]

LEDC_EVT_DUTY_CHNG_END_CH n _EN (n : 0-7) Configures whether to enable the LEDC_EVT_DUTY_CHNG_END_CH n event.

0: Disable

1: Enable

(R/W)

LEDC_EVT_OVF_CNT_PLS_CH n _EN (n : 0-7) Configures whether to enable the LEDC_EVT_OVF_CNT_PLS_CH n event.

0: Disable

1: Enable

(R/W)

LEDC_EVT_TIME_OVF_TIMER_x_EN (x: 0-3) Configures whether to enable the LEDC_EVT_TIME_OVF_TIMER_x event.

0: Disable

1: Enable

(R/W)

LEDC_EVT_TIMER_x_CMP_EN (x: 0-3) Configures whether to enable the LEDC_EVT_TIMER_x_CMP event.

0: Disable

1: Enable

(R/W)

LEDC_TASK_DUTY_SCALE_UPDATE_CH n _EN (n : 0-7) Configures whether to enable the LEDC_TASK_DUTY_SCALE_UPDATE_CH n task.

0: Disable

1: Enable

(R/W)

Register 55.8. LEDC_EVT_TASK_EN1_REG (0x0124)

LEDC_TASK_TIMER3_PAUSE_RESUME_EN								LEDC_TASK_TIMER2_PAUSE_RESUME_EN								LEDC_TASK_TIMER1_PAUSE_RESUME_EN								LEDC_TASK_TIMER0_PAUSE_RESUME_EN							
LEDC_TASK_TIMER3_RST_EN								LEDC_TASK_TIMER2_RST_EN								LEDC_TASK_TIMER1_RST_EN								LEDC_TASK_TIMER0_RST_EN							
LEDC_TASK_OVF_CNT_RST_CH7_EN								LEDC_TASK_OVF_CNT_RST_CH6_EN								LEDC_TASK_OVF_CNT_RST_CH5_EN								LEDC_TASK_OVF_CNT_RST_CH4_EN							
LEDC_TASK_OVF_CNT_RST_CH3_EN								LEDC_TASK_OVF_CNT_RST_CH2_EN								LEDC_TASK_OVF_CNT_RST_CH1_EN								LEDC_TASK_OVF_CNT_RST_CH0_EN							
LEDC_TASK_SIG_OUT_DIS_CH7_EN								LEDC_TASK_SIG_OUT_DIS_CH6_EN								LEDC_TASK_SIG_OUT_DIS_CH5_EN								LEDC_TASK_SIG_OUT_DIS_CH4_EN							
LEDC_TASK_SIG_OUT_DIS_CH3_EN								LEDC_TASK_SIG_OUT_DIS_CH2_EN								LEDC_TASK_SIG_OUT_DIS_CH1_EN								LEDC_TASK_SIG_OUT_DIS_CH0_EN							
LEDC_TASK_TIMER3_CAP_EN								LEDC_TASK_TIMER2_CAP_EN								LEDC_TASK_TIMER1_CAP_EN								LEDC_TASK_TIMER0_CAP_EN							
LEDC_TASK_TIMER3_RES_UPDATE_EN								LEDC_TASK_TIMER2_RES_UPDATE_EN								LEDC_TASK_TIMER1_RES_UPDATE_EN								LEDC_TASK_TIMER0_RES_UPDATE_EN							

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Reset

LEDC_TASK_TIMER x _RES_UPDATE_EN (x : 0-3) Configures whether to enable LEDC_TASK_TIMER x _RES_UPDATE task.
 0: Disable
 1: Enable
 (R/W)

LEDC_TASK_TIMER x _CAP_EN (x : 0-3) Configures whether to enable LEDC_TASK_TIMER x _CAP task.
 0: Disable
 1: Enable
 (R/W)

LEDC_TASK_SIG_OUT_DIS_CH n _EN (n : 0-7) Configures whether to enable LEDC_TASK_SIG_OUT_DIS_CH n task.
 0: Disable
 1: Enable
 (R/W)

LEDC_TASK_OVF_CNT_RST_CH n _EN (n : 0-7) Configures whether to enable LEDC_TASK_OVF_CNT_RST_CH n task.
 0: Disable
 1: Enable
 (R/W)

LEDC_TASK_TIMER x _RST_EN (x : 0-3) Configures whether to enable LEDC_TASK_TIMER x _RST task.
 0: Disable
 1: Enable
 (R/W)

LEDC_TASK_TIMER x _PAUSE_RESUME_EN (x : 0-3) Configures whether to enable LEDC_TASK_TIMER x _PAUSE and LEDC_TASK_TIMER x _RESUME task.
 0: Disable
 1: Enable
 (R/W)

Register 55.9. LEDC_EVT_TASK_EN2_REG (0x0128)

(reserved)																																																																	
LED_C_TASK_GAMMA_RESUME_CH7_EN								LED_C_TASK_GAMMA_RESUME_CH6_EN								LED_C_TASK_GAMMA_RESUME_CH5_EN																																																	
LED_C_TASK_GAMMA_RESUME_CH4_EN								LED_C_TASK_GAMMA_RESUME_CH3_EN								LED_C_TASK_GAMMA_RESUME_CH2_EN																																																	
LED_C_TASK_GAMMA_RESUME_CH1_EN								LED_C_TASK_GAMMA_PAUSE_CH7_EN								LED_C_TASK_GAMMA_PAUSE_CH6_EN																																																	
LED_C_TASK_GAMMA_PAUSE_CH5_EN								LED_C_TASK_GAMMA_PAUSE_CH4_EN								LED_C_TASK_GAMMA_PAUSE_CH3_EN																																																	
LED_C_TASK_GAMMA_PAUSE_CH2_EN								LED_C_TASK_GAMMA_PAUSE_CH1_EN								LED_C_TASK_GAMMA_RESTART_CH7_EN																																																	
LED_C_TASK_GAMMA_RESTART_CH6_EN								LED_C_TASK_GAMMA_RESTART_CH5_EN								LED_C_TASK_GAMMA_RESTART_CH4_EN																																																	
LED_C_TASK_GAMMA_RESTART_CH3_EN								LED_C_TASK_GAMMA_RESTART_CH2_EN								LED_C_TASK_GAMMA_RESTART_CH1_EN																																																	
LED_C_TASK_GAMMA_RESTART_CH0_EN																																																																	
31								24								23		22		21		20		19		18		17		16		15		14		13		12		11		10		9		8		7		6		5		4		3		2		1		0		Reset	
0 0 0 0 0 0 0 0								0 0		0 0 0 0 0 0 0		0 0		0 0 0 0 0 0 0		0 0 0		0 0 0		0 0 0 0 0 0 0		0 0 0		0 0 0 0 0 0 0		0 0 0		0 0 0 0 0 0 0		0 0 0		0 0 0 0 0 0 0		0 0 0		0 0 0 0 0 0 0		0 0 0		0 0 0 0 0 0 0																									

LEDC_TASK_GAMMA_RESTART_CH <i>n</i> _EN (<i>n</i> : 0-7)	Configures	whether	to	enable
LEDC_TASK_GAMMA_RESTART_CH <i>n</i> task.				
0: Disable				
1: Enable				
(R/W)				

LEDC_TASK_GAMMA_PAUSE_CHn_EN (n : 0-7)	Configures	whether	to	enable
LEDC_TASK_GAMMA_PAUSE_CH n task.				
0: Disable				
1: Enable				
(R/W)				

LEDC_TASK_GAMMA_RESUME_CH n _EN (n : 0-7)	Configures whether to enable LEDC_TASK_GAMMA_RESUME_CH n task.
0: Disable	
1: Enable	
(R/W)	

Register 55.10. LEDC_TIMER x _CMP_REG (x : 0-3) (0x0140+0x4* x)

(reserved)												LEDC_TIMER_CMP											
31												0											
0 0 0 0 0 0 0 0 0 0 0 0												0x000											

LEDC_TIMERx_CMP Configures the comparison value for LEDC timer *x*. (R/W)

Register 55.11. LEDC_CONF_REG (0x0170)

Register map diagram for the LEDC module. The register is 32 bits wide, with bits 31 down to 0. Bits 31-10 are reserved. Bits 9-8 are LEDC_CLK_EN. Bits 7-0 are reserved. Bits 9-8 are LEDC_GAMMA_RAM_CLK_EN_CH7. Bits 7-6 are LEDC_GAMMA_RAM_CLK_EN_CH6. Bits 5-4 are LEDC_GAMMA_RAM_CLK_EN_CH5. Bits 3-2 are LEDC_GAMMA_RAM_CLK_EN_CH4. Bit 1 is LEDC_GAMMA_RAM_CLK_EN_CH3. Bit 0 is LEDC_GAMMA_RAM_CLK_EN_CH2. A 'Reset' label is at the bottom right.

LEDC_GAMMA_RAM_CLK_EN_CH n (n : 0-7) Configures whether to open LEDC channel n gamma RAM clock gate.

0: Open the clock gate only when an application writes or reads LEDC channel *n* gamma RAM

1: Force open the clock gate for LEDC channel *n* gamma RAM

(R/W)

LEDC_CLK_EN Configures whether to open the register clock gate.

0: Open the clock gate only when an application writes registers

1: Force open the clock gate for register

(R/W)

Register 55.12. LEDC_CH n _DUTY_R_REG (n : 0-7) (0x0010+0x14* n)

(reserved)							LEDC_DUTY_CH ⁿ _R																									
31	25	24																							0							
0	0	0	0	0	0	0	0	0x000000																								Reset

LEDC_DUTY_CH n _R Represents the current duty cycle of output signal on channel n . (RO)

Register 55.13. LEDC_TIMER_x_VALUE_REG (x: 0-3) (0x00A4+0x8*x)

Register 0x00000000 (LED_TIMER_CNT) bit field diagram. The register is 32 bits wide, divided into a 12-bit reserved field (bits 31-20) and a 20-bit LED_TIMER_CNT field (bits 19-0). The LED_TIMER_CNT field contains the value 0.

LEDC_TIMER x _CNT Represents the current counter value of timer x . (RO)

Register 55.14. LEDC_TIMER_x_CNT_CAP_REG (x: 0-3) (0x0150+0x4*x)

Register structure diagram for LEDC_TIMER_CNT_CAP. The register is 32 bits wide, divided into a 12-bit reserved field (bits 31-20) and a 20-bit data field (bits 19-0). The data field contains the value 0x000. A red 'X' marks the CNT_CAP bit in the data field. A 'Reset' label is at the bottom right.

LEDC_TIMERx_CNT_CAP Represents the captured LEDC timer **x** count value. (RO)

Register 55.15. LEDC_INT_RAW_REG (0x00C0)

[illegible]

LEDC_TIMER_x_OVF_INT_RAW (x: 0-3) Raw status bit: The raw interrupt status of LEDC_TIMER_x_OVF_INT. Triggered when the timer **x** has reached its maximum counter value. (R/WTC/SS)

LEDC_DUTY_CHNG_END_CH n _INT_RAW (n : 0-7) Raw status bit: The raw interrupt status of LEDC_DUTY_CHNG_END_CH n _INT. Triggered when the fading of duty has finished. (R/WTC/SS)

LEDC_OVF_CNT_CH*n*_INT_RAW (*n*: 0-7) Raw status bit: The raw interrupt status of LEDC_OVF_CNT_CH*n*_INT. Triggered when the ovf_cnt has reached the value specified by LEDC_OVF_NUM_CH*n*. (R/WTC/SS)

Register 55.16. LEDC_INT_ST_REG (0x00C4)

(reserved)												LEDC_OVF_CNT_CH7_INT_ST LEDC_OVF_CNT_CH6_INT_ST LEDC_OVF_CNT_CH5_INT_ST LEDC_OVF_CNT_CH4_INT_ST LEDC_OVF_CNT_CH3_INT_ST LEDC_OVF_CNT_CH2_INT_ST LEDC_OVF_CNT_CH1_INT_ST LEDC_DUTY_CHNG_END_CH0_INT_ST LEDC_DUTY_CHNG_END_CH7_INT_ST LEDC_DUTY_CHNG_END_CH6_INT_ST LEDC_DUTY_CHNG_END_CH5_INT_ST LEDC_DUTY_CHNG_END_CH4_INT_ST LEDC_DUTY_CHNG_END_CH3_INT_ST LEDC_DUTY_CHNG_END_CH2_INT_ST LEDC_TIMER3_OVF_INT_ST LEDC_TIMER2_OVF_INT_ST LEDC_TIMER1_OVF_INT_ST LEDC_TIMER0_OVF_INT_ST																						
3120												19	18	17		16	15	14	13	12	11		10	9	8	7	6	5	4	3	2	1	0	
000000000000												0	0	0		0	0	0	0	0	0		0	0	0	0	0	0	0	0	0	0	0	Reset

LEDC_TIMER x _OVF_INT_ST (x : 0-3) Masked status bit: The masked interrupt status of LEDC_TIMER x _OVF_INT. Valid only when LEDC_TIMER x _OVF_INT_ENA is set to 1. (RO)

LEDC_DUTY_CHNG_END_CH n _INT_ST (n : 0-7) Masked status bit: The masked interrupt status of LEDC_DUTY_CHNG_END_CH n _INT. Valid only when LEDC_DUTY_CHNG_END_CH n _INT_ENA is set to 1. (RO)

LEDC_OVF_CNT_CH n _INT_ST (n : 0-7) Masked status bit: The masked interrupt status of LEDC_OVF_CNT_CH n _INT. Valid only when LEDC_OVF_CNT_CH n _INT_ENA is set to 1. (RO)

Register 55.17. LEDC_INT_ENA_REG (0x00C8)

(reserved)												LEDC_OVF_CNT_CH7_INT_ENA LEDC_OVF_CNT_CH6_INT_ENA LEDC_OVF_CNT_CH5_INT_ENA LEDC_OVF_CNT_CH4_INT_ENA LEDC_OVF_CNT_CH3_INT_ENA LEDC_OVF_CNT_CH2_INT_ENA LEDC_OVF_CNT_CH1_INT_ENA LEDC_DUTY_CHNG_END_CH0_INT_ENA LEDC_DUTY_CHNG_END_CH7_INT_ENA LEDC_DUTY_CHNG_END_CH6_INT_ENA LEDC_DUTY_CHNG_END_CH5_INT_ENA LEDC_DUTY_CHNG_END_CH4_INT_ENA LEDC_DUTY_CHNG_END_CH3_INT_ENA LEDC_DUTY_CHNG_END_CH2_INT_ENA LEDC_TIMER3_OVF_INT_ENA LEDC_TIMER2_OVF_INT_ENA LEDC_TIMER1_OVF_INT_ENA LEDC_TIMER0_OVF_INT_ENA															
3120												1918	171615141312	1110	9876543210	Reset											
000000000000												00	00000000	00	000000000000000000												

LEDC_TIMER x _OVF_INT_ENA (x : 0-3) Enable bit: Write 1 to enable LEDC_TIMER x _OVF_INT. (R/W)

LEDC_DUTY_CHNG_END_CH n _INT_ENA (n : 0-7) Enable bit: Write 1 to enable LEDC_DUTY_CHNG_END_CH n _INT. (R/W)

LEDC_OVF_CNT_CH n _INT_ENA (n : 0-7) Enable bit: Write 1 to enable LEDC_OVF_CNT_CH n _INT. (R/W)

Register 55.18. LEDC_INT_CLR_REG (0x00CC)

(reserved)																LEDC_OVF_CNT_CH7_INT_CLR LEDC_OVF_CNT_CH6_INT_CLR LEDC_OVF_CNT_CH5_INT_CLR LEDC_OVF_CNT_CH4_INT_CLR LEDC_OVF_CNT_CH3_INT_CLR LEDC_OVF_CNT_CH2_INT_CLR LEDC_OVF_CNT_CH1_INT_CLR LEDC_DUTY_CHNG_END_CH0_INT_CLR LEDC_DUTY_CHNG_END_CH7_INT_CLR LEDC_DUTY_CHNG_END_CH6_INT_CLR LEDC_DUTY_CHNG_END_CH5_INT_CLR LEDC_DUTY_CHNG_END_CH4_INT_CLR LEDC_DUTY_CHNG_END_CH3_INT_CLR LEDC_DUTY_CHNG_END_CH2_INT_CLR LEDC_TIMER3_OVF_INT_CLR LEDC_TIMER2_OVF_INT_CLR LEDC_TIMER1_OVF_INT_CLR LEDC_TIMER0_OVF_INT_CLR																			
3120																1918	171615141312	1110	9876543210	Reset															
00000000000000																00	00000000	00	000000000000000000000000000000																

LEDC_TIMER_x_OVF_INT_CLR (**x**: 0-3) Clear bit: Write 1 to clear LEDC_TIMER_x_OVF_INT. (WT)

LEDC_DUTY_CHNG_END_CH_n_INT_CLR (**n**: 0-7) Clear bit: Write 1 to clear LEDC_DUTY_CHNG_END_CH_n_INT. (WT)

LEDC_OVF_CNT_CH_n_INT_CLR (**n**: 0-7) Clear bit: Write 1 to clear LEDC_OVF_CNT_CH_n_INT. (WT)

Register 55.19. LEDC_DATE_REG (0x0174)

(reserved)				LEDC_LEDC_DATE																										
31	28	27																										0		
0	0	0	0	0x2303070																										Reset

LEDC_LEDC_DATE Configures the version. (R/W)